

ECOM EXPERTS

Ecom relay

Manual de usuario, técnico y de desarrollo

ecom_relay · 08/09/2026

Contenido

Antes de empezar

Instalación del módulo

Configuración

Probar que funciona

Mensajes de error frecuentes

Preguntas frecuentes

1. Propósito

2. Arquitectura

3. Protocolo cliente ↔ relay

Petición (módulo → relay)

Respuesta (relay → módulo)

4. Seguridad

5. Rotación de IP

6. Ficheros del módulo

7. Configuración (Configuration::get)

8. Compatibilidad

1. Cargar el helper

2. API

Ecom_RelayClient::request(array \$opts): array

Ecom_RelayClient::requestJson(array \$opts): array

Ecom_RelayClient::isEnabled(): bool

3. Ejemplo: llamada a Gemini a través del relay

4. Patrón de adopción recomendado

5. Fallback y errores

6. Buenas prácticas

Este módulo hace que las llamadas a APIs externas de tu tienda salgan por un **servidor relay con varias IP**, en lugar de por la IP de tu hosting. Así proteges la reputación de la IP de tu tienda y evitas bloqueos/limitaciones.

Antes de empezar

Necesitas tener el **servidor relay** instalado y funcionando en un VPS (ver la guía del servidor, [server/README.md](#)). De ahí necesitas dos datos: 1. La **URL del relay** (ej. <https://relay.tudominio.com/>). 2. La **clave secreta** configurada en el servidor.

Instalación del módulo

1. Sube la carpeta `ecom_relay` a `/modules/` de tu PrestaShop (o instala el ZIP).
2. Ve a **Módulos** → busca "Ecom Relay" → **Instalar**.
3. Pulsa **Configurar**.

Configuración

Campo	Qué poner
Activar relay	Sí (cuando ya tengas URL y clave)
URL del relay	La URL HTTPS del servidor relay
Clave secreta compartida	La misma clave del servidor. (Vacío = no cambiar)
Timeout	60 (segundos) suele bastar
Fallback directo	Sí (recomendado): si el relay está caído, la llamada se hace directa para no romper la tienda
Depuración (logs)	Actívalo solo si necesitas diagnosticar
URL de prueba	Por defecto https://api.ipify.org/?format=json

Pulsa **Guardar**.

Probar que funciona

Pulsa el botón **Probar conexión**. Si todo está bien verás un mensaje verde con: - **Via:** `relay` (si saliera `direct`, el relay no respondió). - **IP de salida:** la IP con la que el relay hizo la llamada. - **Status destino:** 200.

Para que la prueba con `api.ipify.org` funcione, ese dominio debe estar en la whitelist del servidor relay. Si ves `WHITELIST_DENIED`, pídele a quien gestione el servidor que lo añada, o cambia la "URL de

prueba" por un dominio ya permitido.

Mensajes de error frecuentes

Mensaje	Causa / Solución
Relay [AUTH]: Firma invalida	La clave del módulo no coincide con la del servidor, o el reloj del servidor está desajustado (NTP).
Relay [WHITELIST_DENIED]	El dominio destino no está permitido en el servidor.
No se pudo contactar el relay	URL incorrecta, servidor caído o sin HTTPS válido.
Relay [RATE_LIMIT]	Demasiadas peticiones; sube el límite en el servidor.

Preguntas frecuentes

¿Esto acelera mis APIs? No; su objetivo es **proteger la IP** de la tienda y repartir la carga entre varias IP de salida.

¿Pasa todo mi tráfico por aquí? Solo el de los módulos que usen este relay (los que llamen al helper). No afecta a la navegación normal de la tienda.

¿Es seguro? Sí: cada petición va firmada, el servidor solo reenvía a dominios que estén en su lista blanca y bloquea destinos internos. Usa siempre HTTPS.

¿Y si desactivo el relay? Con "Fallback directo" en Sí, las llamadas se harán directas (como antes). Con No, devolverán error para que te enteres.

Ecom Relay — Documentación Técnica

1. Propósito

Enrutar las llamadas HTTP salientes de la tienda PrestaShop a través de un servidor relay que rota la IP de salida, protegiendo la reputación de la IP del hosting de la tienda frente a rate-limits/bloqueos de las APIs externas (IA y otras).

2. Arquitectura

```
[Módulo X] -uses-> Ecom_RelayClient::request()
                    | firma HMAC + envelope JSON
                    ▼ HTTPS POST
[Servidor Relay /public/index.php]
├ Relay_Auth      (HMAC + timestamp)
├ Relay_RateLimiter (ventana fija)
├ Relay_Whitelist (dominios)
├ Relay_Ssrf      (IP privadas)
├ Relay_IpPool    (round_robin/random)
└ Relay_Forwarder (cURL + CURLOPT_INTERFACE)
    ▼
[API destino]
```

3. Protocolo cliente ↔ relay

Petición (módulo → relay)

- `POST` a la URL del relay, `Content-Type: application/json`.
- Cabeceras de firma:
- `X-Relay-Timestamp`: epoch en segundos.
- `X-Relay-Signature`: `hmac_sha256(timestamp + "\n" + body, shared_secret)`.
- Cuerpo (envelope):

```
{
  "url": "https://destino/...",
  "method": "POST",
  "headers": { "Content-Type": "application/json" },
  "body_b64": "<base64 del cuerpo>",
  "timeout": 60
}
```

Respuesta (relay → módulo)

```
{
  "ok": true,
  "status": 200,
  "headers": { "Content-Type": "application/json" },
  "body_b64": "<base64 de la respuesta>",
  "egress_ip": "203.0.113.11",
  "duration_ms": 412
}
```

En error: { "ok": false, "code": "WHITELIST_DENIED", "error": "..." }.

4. Seguridad

- **Autenticación HMAC-SHA256 con timestamp** (anti-replay, tolerancia configurable). Comparación en tiempo constante (`hash_equals`).
- **Whitelist de dominios** con comodín de subdominio. Por defecto deniega.
- **Anti-SSRF**: resuelve el host y rechaza IP privadas/reservadas (`FILTER_FLAG_NO_PRIV_RANGE` | `FILTER_FLAG_NO_RES_RANGE`).
- **Sin redirecciones** por defecto; `CURLPROTO_HTTP|HTTPS` ; verificación TLS estricta.
- **Rate-limit**, `max_timeout` , `max_body_bytes` .
- Carpetas `config/` , `src/` , `storage/` fuera del webroot + `.htaccess` de denegación.

5. Rotación de IP

`Relay_IpPool::pick()` elige una IP del pool del VPS. `Relay_Forwarder` la aplica con `curl_setopt($ch, CURLOPT_INTERFACE, $ip)` , forzando que la conexión salga por esa IP. `round_robin` persiste un contador con `flock` ; `random` usa `random_int` .

6. Ficheros del módulo

Fichero	Rol
<code>ecom_relay.php</code>	Clase principal <code>Ecom_Relay</code> , configuración, prueba, registro Gmartos
<code>classes/Ecom_RelayClient.php</code>	Helper reutilizable (firma, reenvío, fallback)
<code>classes/Ecom_RelayLogger.php</code>	Logger condicional (<code>ECOM_RELAY_DEBUG</code>)
<code>views/templates/admin/gmartos_update.tpl</code>	Aviso de nueva versión

7. Configuración (Configuration::get)

Clave	Defecto	Descripción
ECOM_RELAY_ENABLED	0	Activa el enrutado por relay
ECOM_RELAY_URL	"	URL del servidor relay (HTTPS)
ECOM_RELAY_SECRET	"	Clave compartida (HMAC)
ECOM_RELAY_TIMEOUT	60	Timeout por defecto (s)
ECOM_RELAY_FALLBACK_DIRECT	1	Llamada directa si el relay está inalcanzable
ECOM_RELAY_DEBUG	0	Logs en <code>/logs</code>
ECOM_RELAY_TEST_URL	api.ipify.org	Destino del botón "Probar"

8. Compatibilidad

- PrestaShop 1.7 / 8 / 9. PHP 7.4–8.x.
- Sin Composer ni Guzzle: cURL nativo. Sin namespace en la clase principal.
- Registro Gmartos con cURL nativo (timeout 3s, ping semanal, fallo silencioso).
- Nuevas claves se autocrean vía `addnewfeatures()` al abrir la configuración (sin upgrade files).

Ecom Relay — Guía de Desarrollo / Integración

Cómo usar el helper `Ecom_RelayClient` desde otros módulos para que sus llamadas salientes pasen por el relay (IP rotada).

1. Cargar el helper

```
require_once _PS_MODULE_DIR_ . 'ecom_relay/classes/Ecom_RelayClient.php';
```

```
Comprueba antes que el módulo está instalado, p. ej.: if (!class_exists('Ecom_RelayClient')) { /*
relay no disponible, usa tu cURL directo */ }
```

2. API

`Ecom_RelayClient::request(array $opts): array`

Opciones de entrada:

Clave	Tipo	Defecto	Descripción
<code>url</code>	string	—	URL destino (obligatoria)
<code>method</code>	string	<code>GET</code>	GET/POST/PUT/PATCH/DELETE
<code>headers</code>	array	<code>[]</code>	Mapa <code>['Nombre'=>'valor']</code> o lista <code>['Nombre: valor']</code>
<code>body</code>	string	<code>''</code>	Cuerpo crudo (ej. JSON ya serializado)
<code>timeout</code>	int	config	Segundos

Devuelve:

```
[
  'ok'      => bool,    // true si hubo respuesta del destino
  'status'  => int,     // código HTTP del destino
  'headers' => array,   // cabeceras de respuesta
  'body'    => string,  // cuerpo de respuesta (ya decodificado de base64)
  'error'   => ?string, // mensaje si ok=false
  'egress_ip' => ?string, // IP de salida usada por el relay
  'via'     => string,  // 'relay' | 'direct' | 'none'
]
```

Ecom_RelayClient::requestJson(array \$opts): array

Igual que `request()` pero fuerza `Content-Type: application/json` y añade `'json'` con el cuerpo ya decodificado (`json_decode(..., true)` o `null`).

Ecom_RelayClient::isEnabled(): bool

`true` si el relay está activo y configurado (URL + secreto).

3. Ejemplo: llamada a Gemini a través del relay

```
require_once _PS_MODULE_DIR_ . 'ecom_relay/classes/Ecom_RelayClient.php';

$apiKey = Configuration::get('MI_MODULO_GEMINI_KEY');
$model = 'gemini-flash-latest';
$url = 'https://generativelanguage.googleapis.com/v1beta/models/'
    . $model . ':generateContent?key=' . $apiKey;

$payload = json_encode([
    'contents' => [['parts' => [['text' => 'Hola']]]],
]);

$res = Ecom_RelayClient::requestJson([
    'url' => $url,
    'method' => 'POST',
    'body' => $payload,
    'timeout' => 60,
]);

if ($res['ok'] && $res['status'] === 200) {
    $data = $res['json'];
    // ... usar $data ...
} else {
    PrestaShopLogger::addLog('Relay/Gemini error: ' . $res['error'], 3);
}
```

Recuerda añadir `generativelanguage.googleapis.com` (o `*.googleapis.com`) a la whitelist del servidor relay (`config/config.php`).

4. Patrón de adopción recomendado

Crea en tu módulo un pequeño wrapper que use el relay si está disponible y, si no, caiga a tu cURL directo de siempre. Así desacoplas tus módulos del relay:

```
function mi_modulo_http_post($url, $body, array $headers = [])
{
    $relay = _PS_MODULE_DIR_ . 'ecom_relay/classes/Ecom_RelayClient.php';
    if (file_exists($relay)) {
        require_once $relay;
        if (Ecom_RelayClient::isEnabled()) {
            return Ecom_RelayClient::request([
                'url' => $url, 'method' => 'POST',
                'headers' => $headers, 'body' => $body,
            ]);
        }
    }
    // ... fallback: tu cURL directo, devolviendo el mismo formato de array ...
}
```

5. Fallback y errores

- `ECOM_RELAY_FALLBACK_DIRECT=1`: si el relay está **inalcanzable** (error de transporte), la llamada se hace directa automáticamente. Las **denegaciones lógicas** del relay (`AUTH` , `WHITELIST_DENIED` , `SSRF` , `RATE_LIMIT`) **no** usan fallback: se devuelven como `ok=false` para que el problema sea visible.
- Activa `ECOM_RELAY_DEBUG` para registrar el detalle en `/modules/ecom_relay/logs/`.

6. Buenas prácticas

- No metas el `?key=` ni cabeceras de API key en logs.
- Trata `status` del destino (4xx/5xx) en tu lógica; `ok=true` solo significa que hubo respuesta.
- Para binarios (imágenes), el `body` ya viene decodificado; úsalo tal cual.